第2回 演習: LoRA で回答形式を学習し、学習前後を比較する

架空の講座 FAQ を題材に、小さな指示チューニング済みモデルを LoRA で短時間学習し、学習の前後で **何が変わり、何が変わらないか** を比べます。評価は **形式遵守率**（`answer` と `needs_review` の JSON を守れたか）と **内容正解率** に分けます。

この演習で確かめること

1. プロンプトで形式を指示しても、小さなモデルは守れないことがある（セクション 3、学習前）。
2. LoRA で数分学習すると、形式はほぼ守れるようになる（セクション 5、学習後）。
3. 形式が整っても、FAQ に無いことを聞かれたときの判断（確認を求めるか、断定してしまうか）までは保証されない（セクション 5、6）。
4. 発展: 同じモデルを 4bit 量子化で読み込むと、学習しなくても出力が変わる（セクション 8）。

進め方

セクション	内容	所要時間の目安（T4）
0〜3	環境、設定、データ生成、モデル読み込み、学習前評価	約 5 分
4〜7	学習、学習後評価、知識質問、比較表と提出	約 10 分（学習 4〜5 分を含む）
8（任意）	4bit 量子化で読み込んで比べる	約 2 分

授業前に 0〜3 を実行して環境が動くことを確かめておくと、当日の詰まりが減ります（任意）。Colab はセッションが切れると変数が消えるので、その場合は当日あらためて 0 から実行します。

使うもの: Google Colab（T4 GPU、無料枠）。モデルは `Qwen/Qwen3.5-0.8B`（Apache-2.0）。データはこのノートブックが生成する架空データで、実在の締切や URL は含みません。

読み方: 各セクションの説明に「やること」「見るところ」「うまくいかないとき」を書いています。セルは上から順に実行してください。途中のセルを飛ばすと、後のセルで変数が見つからないエラーになります。

版: 1.1（2026-10-18）

0. 環境の確認とライブラリの導入

やること: GPU が割り当てられているかを確認し、ライブラリを動作確認済みの版に固定して導入します。

見るところ

- 1 つ目のセルに `Tesla T4` と表示されること。
- 2 つ目のセルの最後に、版の一覧（`transformers: 5.18.0` など）が表示されること。導入は 1 分ほどかかります。

うまくいかないとき

- 「GPU が見つかりません」と出る: メニューの「ランタイム」→「ランタイムのタイプを変更」→「T4 GPU」を選び、セルを実行し直します。混雑で割り当てられないときは、時間をおいて試します。
- pip の赤い `ERROR: pip's dependency resolver ...`（`cudf` や `pyarrow` に関するもの）は、Colab に最初から入っている別のパッケージとの食い違いの警告です。この演習には影響しないので、そのまま進めます。

```
import subprocess
try:
    gpu_info = subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total", "
except FileNotFoundError:   # CPU ランタイムには nvidia-smi が無い
    gpu_info = ""
print(gpu_info or "GPU が見つかりません。ランタイム → ランタイムのタイプを変更
```

```
# Colab の T4（Python 3.13、torch 2.11.0+cu130）で動作確認した版。torch は Colal
PINS = {
    "transformers": "5.18.0",   # Qwen3.5 は transformers 5 系が必要
    "peft": "0.21.1",           # LoRA の実装
    "trl": "1.14.2",            # 学習ループ（SFTTrainer）
    "accelerate": "1.15.0",
    "datasets": "5.1.0",
}
spec = " ".join(f'"{k}=={v}"' for k, v in PINS.items())
%pip uninstall -y -q torchao   # Colab 同梱の torchao（0.10）は peft 0.21 と非互
%pip install -q {spec}

# cudf-cu13 と pyarrow の依存警告が ERROR 表示されるが、cudf は使わないので無視
import transformers, peft, trl, accelerate, datasets, torch
print({m.__name__: m.__version__ for m in [transformers, peft, trl, accelerate,
```

1. 設定

やること: モデル名、学習のエポック数（データを何周するか）などを 1 つの辞書 `CONFIG` にまとめます。この辞書はセクション 7 の提出用ファイルにそのまま記録されるので、「どの条件で測ったか」が後から分かります。

- `revision` を固定しているのは、いつ実行しても同じ重みを使うためです。学習前後の比較や、他の人との比較で条件が揃います。

- `num_train_epochs: 20` は、学習データ（14 件）を 20 周する設定です。1 周が 2 ステップなので合計 40 ステップで、T4 で学習が 4〜5 分に収まります。
- `gen: {"do_sample": False}` は貪欲生成です。学習前後で生成条件を揃え、揺れの影響を除きます。

`USE_DRIVE` を `True` にすると Google Drive をマウントし、学習した adapter と結果ファイルを Drive にも保存します。セッションが切れても結果を残したいときに使います。1 回のセッションで完結させるなら `False` のままで構いません。

見るところ: 設定の JSON が表示されます。

```
from pathlib import Path
import os, json, time

CONFIG = {
    "base_model": "Qwen/Qwen3.5-0.8B",    # 小さな指示チューニング済みモデル（Apa
    "revision": "2fc06364715b967f1860aea9cf38778875588b17",  # いつ実行しても同
    "seed": 42,
    "num_train_epochs": 20,  # 学習データ（14 件）を何周学習するか。1 周 2 ステ
    "warmup_steps": 6,
    "lr": 2e-4,
    "lora_r": 16,            # LoRA の行列の次元（小さいほど学習するパラメータカ
    "lora_alpha": 32,
    "max_seq_len": 512,
    "max_new_tokens": 96,
    "gen": {"do_sample": False},    # 評価は貪欲生成で固定（前後比較の条件を揃え
    "work_dir": "/content/qlora_lab",                          # 結果ファイルの
    "drive_dir": "/content/drive/MyDrive/aie2026/qlora_lab",  # USE_DRIVE=True
}
WORK = Path(CONFIG["work_dir"]); WORK.mkdir(parents=True, exist_ok=True)

USE_DRIVE = False  # True にすると Google Drive をマウントして adapter と出力を
if USE_DRIVE:
    from google.colab import drive
    drive.mount("/content/drive")
    Path(CONFIG["drive_dir"]).mkdir(parents=True, exist_ok=True)
print(json.dumps(CONFIG, ensure_ascii=False, indent=2))
```

2. 架空 FAQ と教師データの生成

やること: 5 項目の架空 FAQ から、学習用（train）、検証用（valid）、評価用（test）のデータを作ります。

モデルへの入力は「FAQ ＋ 質問」、望ましい出力は `{"answer": <文字列>, "needs_review": <真偽値>}` の JSON 1 行です。

- FAQ に根拠がある質問には、FAQ の文で答え、`needs_review: false` にする。
- FAQ に無い質問には、「この FAQ には記載がありません。運営に確認が必要です。」と答え、`needs_review: true` にする。

データの分け方

- 同じ FAQ 項目を聞く質問の言い換えを、**テンプレート単位**で train / valid / test に振り分けます。学習で見た文がそのまま test に出ないようにするためです。
- test は 21 問（根拠あり 15、根拠なし 6）です。件数を確保するため、test にだけ使う質問を別に用意しています。
- 正解（`needs_review` の値と、答えに含まれるべき語句）は `gold` に分けて持ち、モデルには渡しません。
- train は 14 件（根拠あり 10、根拠なし 4）、valid は 7 件です。FAQ が 5 項目しかないので学習データは少なく、同じ例を何回も学習することになります。何回学習するかは、データを複製して増やすのではなく、セクション 4 のエポック数（`num_train_epochs`）で決めます。データのファイルには実例だけを入れ、学習量は学習の設定で持つのが基本です。

見るところ: `train=14 valid=7 test=21 gold=21` と、学習例の 1 件（質問と、望ましい出力）が表示されます。質問が train / valid / test のどれかで重複していると、ここでエラーになります。

```
FAQ = {
    "deadline": "課題の提出期限は 9 月 30 日 23:59（日本時間）です。",
    "destination": "提出先は学習ポータルの「課題」欄です。",
    "format": "提出形式は PDF 1 ファイルで、ファイル名は 学籍番号_課題番号.pdf
    "office_hours": "質問は毎週火曜 17:00〜18:00 のオフィスアワーで受け付けます。
    "late": "期限後の提出は、期限から 48 時間以内なら減点のうえ受け付けます。",
}
FAQ_TEXT = "¥n".join(f"- {v}" for v in FAQ.values())

# 根拠あり:（FAQ キー，質問テンプレート群，正解の answer に含まれるべき語）
GROUNDED = {
    "deadline": (["課題の提出期限はいつですか", "いつまでに出せばいいですか", "
    "destination": (["提出先はどこですか", "どこにアップロードしますか", "課題は
    "format": (["提出形式を教えてください", "ファイル名の決まりはありますか", "
    "office_hours": (["質問はいつできますか", "オフィスアワーはいつですか", "先
    "late": (["遅れて出したらどうなりますか", "期限を過ぎた場合は受け付けますか"
}
# 根拠なし: FAQ に無いので needs_review=true が正解
UNGROUNDED = [
    "再提出はできますか", "欠席した回の扱いを教えてください", "他の受講者の点数
    "成績はいつ出ますか", "提出期限は延長できますか", "グループでの提出は可能で
    "課題の配点はどうなっていますか", "講義の録画は公開されますか",
]

# test 専用の追加質問（train / valid には入れない）
TEST_EXTRA_GROUNDED = {
    "deadline": ["提出期限の日付と時刻を教えてください", "課題の締切は何月何日の
    "destination": ["課題の提出はどこで行いますか", "提出物を出す先を教えてくだ
    "format": ["提出するファイルの形式は何ですか", "提出ファイルは何形式で、いく
    "office_hours": ["オフィスアワーの曜日と時間を教えてください", "質問を受け付
    "late": ["提出が締切に間に合わなかったらどうなりますか", "期限後に提出した課
}
```

```
TEST_EXTRA_UNGROUNDED = [
    "課題の文字数に制限はありますか", "提出した課題へのフィードバックはもらえま
    "参考文献の書き方に指定はありますか", "オフィスアワーは事前予約が必要ですか"
]
UNGROUNDED_ANSWER = "この FAQ には記載がありません。運営に確認が必要です。"

def make_example(question: str, answer: str, needs_review: bool) -> dict:
    target = json.dumps({"answer": answer, "needs_review": needs_review}, ensure
    return {"question": question, "faq": FAQ_TEXT, "target": target, "needs_rev

def split_templates(templates):
    # テンプレートの先頭 2 つを train、3 つ目を valid、4 つ目を test に固定（言
    return templates[:2], templates[2:3], templates[3:4]

train, valid, test, gold = [], [], [], {}
for key, (templates, must_include) in GROUNDED.items():
    tr, va, te = split_templates(templates)
    for q in tr: train.append(make_example(q, FAQ[key], False))
    for q in va: valid.append(make_example(q, FAQ[key], False))
    for q in te + TEST_EXTRA_GROUNDED[key]:
        test.append(make_example(q, FAQ[key], False)); gold[q] = {"needs_review"
for i, q in enumerate(UNGROUNDED):
    ex = make_example(q, UNGROUNDED_ANSWER, True)
    bucket = [train, train, valid, test][i % 4]   # 根拠なしの質問は 4 問ごとに
    bucket.append(ex)
    if bucket is test: gold[q] = {"needs_review": True, "must_include": ["確認"]
for q in TEST_EXTRA_UNGROUNDED:
    test.append(make_example(q, UNGROUNDED_ANSWER, True)); gold[q] = {"needs_rev

# train は 14 件（根拠あり 10、根拠なし 4）。同じ例を何回学習するかは、データを
all_questions = [r["question"] for r in train + valid + test]
assert len(set(all_questions)) == len(all_questions), "質問が train / valid / te
print(f"train={len(train)} valid={len(valid)} test={len(test)} gold={len(gold)}")
print("学習例の 1 件 | 質問:", train[0]["question"], "| 望ましい出力:", train[0]
_ = (WORK / "train.jsonl").write_text("¥n".join(json.dumps(r, ensure_ascii=Fals
_ = (WORK / "test.jsonl").write_text("¥n".join(json.dumps(r, ensure_ascii=False)
_ = (WORK / "gold.json").write_text(json.dumps(gold, ensure_ascii=False, indent=
```

3. モデルの読み込みと学習前評価

やること: 基盤モデルを量子化なし（32bit）で読み込み、test の 21 問に答えさせて採点します。

プロンプト（`SYSTEM`）には「必ず JSON 1 行で、`answer` と `needs_review` を返す」と書いてあります。つまり **形式は指示済み** です。このモデルがその指示を守れるかどうかが、学習前の確認点です。

採点のしかた（セクション 5 でも同じ関数を使います）

- **形式**: JSON として読め、キーが `answer` と `needs_review` のちょうど 2 つで、型が文字列と真偽値であること。

- **内容**: `needs_review` が正解と一致し、かつ正解の語句（例: 「9 月 30 日」と「23:59」）が `answer` に含まれること。

内容の判定には `needs_review` の一致が必要です。そのため、キーが欠けている出力は、文面が正しくても内容は × になります。文面そのものは、表で自分の目で確かめてください。

`Qwen/Qwen3.5-0.8B` は画像も扱えるモデルですが、`AutoModelForCausalLM` で読み込むとテキスト部分だけが使われます。

見るところ

- 1 つ目のセル: `重みのメモリ(GB): 3.01` のように表示されます。モデルのダウンロード（約 1.75GB）に 1 分ほどかかります。
- 2 つ目のセル: 1 行の要約（`format_rate`、`content_rate` など）と、21 問の出力の表。評価には 1 分前後かかります。学習前は、多くの出力で `needs_review` が欠けているはずです。

出ても問題ない警告

- `You are sending unauthenticated requests to the HF Hub`: Hugging Face にログインせずにモデルを取りに行っている、という案内です。公開モデルはログインしなくてもダウンロードできます。ログインが要るのは次の場合で、モデルの大きさでは決まりません。
  - 利用規約への同意が必要なモデル（gated model。Llama や Gemma など）や、非公開のリポジトリを使うとき。
  - 自分のモデルやデータを Hub にアップロードするとき。
  - 短時間に大量のリクエストを送り、回数の上限に当たるとき。上限はログインしていないと IP アドレスごとに 5 分あたり 3,000 回のファイル取得（2025 年 9 月時点の公表値。Hub の rate limits）で、ログインすると上がります。

  このノートブックのモデルは公開されていて同意も不要です。読み込みで送るリクエストは多くても数十回なので、上限には届きません。もし上限に当たって `429 Too Many Requests` が出ても、ライブラリが待ち時間を読み取って自動で再試行します。ログインが必要になったときは、Colab の左側の鍵アイコン（シークレット）に Hugging Face のトークンを `HF_TOKEN` という名前で登録し、このノートブックからのアクセスを許可すると、自動で使われます。
- `... is falling back to its reference PyTorch implementation ...`: このモデルの一部の層（線形注意）に対する高速な実装が入っていないので、遅いが正しい実装で動くという案内です。4 行ほど出ます。

ここで読んでおくこと: 表を見て、「どのキーが欠けているか」「文面は FAQ と合っているか」を確認します。この表はセクション 5 で学習後の出力と並べて見ます。出力の

原文は /content/qlora_lab/eval_before.json にも保存されています（左のファイル一覧から開けます）。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

DEVICE = "cuda" if torch.cuda.is_available() else ("mps" if torch.backends.mps.
tok = AutoTokenizer.from_pretrained(CONFIG["base_model"], revision=CONFIG["revi
if tok.pad_token is None: tok.pad_token = tok.eos_token
# 量子化なし。重みは 32bit のまま保持する（dtype を省くと配布時の精度で読み込ま
model = AutoModelForCausalLM.from_pretrained(CONFIG["base_model"], revision=CONI
model.config.use_cache = True
BASE_MEMORY_GB = round(model.get_memory_footprint() / 1e9, 2)
print("loaded:", CONFIG["base_model"], "| device:", DEVICE, "| 重みのメモリ(GB)

# 形式はプロンプトで指示している。モデルがこれを守れるかが学習前の確認点
SYSTEM = "あなたは講座 FAQ の回答係です。与えられた FAQ だけを根拠に、必ず JSON

def build_messages(example):
    user = f"FAQ:¥n{example['faq']}¥n¥n質問: {example['question']}"
    return [{"role": "system", "content": SYSTEM}, {"role": "user", "content": u

def generate(model, example):
    # enable_thinking=False: 思考ブロックを空にして回答だけを生成させる（モデル
    prompt = tok.apply_chat_template(build_messages(example), tokenize=False, ad
    inputs = tok(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        out = model.generate(**inputs, max_new_tokens=CONFIG["max_new_tokens"],
    return tok.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens
```

```
import re
import pandas as pd
pd.set_option("display.max_colwidth", None)   # 出力の原文を省略せずに表示する

def check_format(text: str):
    '''JSON として読め、必須キーと型が合うか。失敗理由も返す'''
    try:
        obj = json.loads(text)
    except Exception as e:
        return False, f"json_error: {type(e).__name__}", None
    if not isinstance(obj, dict): return False, "not_object", None
    if set(obj) - {"answer", "needs_review"}: return False, "extra_keys", obj
    if "answer" not in obj or "needs_review" not in obj: return False, "missing_
    if not isinstance(obj["answer"], str) or not isinstance(obj["needs_review"],
    return True, "", obj

def check_content(obj, gold_entry):
    '''内容の合否: needs_review が正解と一致し、必須語句を answer に含む'''
    if obj is None: return False
    if obj.get("needs_review") != gold_entry["needs_review"]: return False
    return all(w in obj.get("answer", "") for w in gold_entry["must_include"])

def mark(ok: bool) -> str:
```

```
    return "○" if ok else "×"

def evaluate(model, label: str):
    '''test の 21 問を生成して採点し、要約と質問ごとの結果を返す。原文は eval_<
    rows = []
    t0 = time.time()
    for ex in test:
        raw = generate(model, ex)
        ok_fmt, reason, obj = check_format(raw)
        # 形式が壊れていても JSON らしい部分が読めるなら、内容だけは別に採点する
        loose = obj
        if not ok_fmt:
            m = re.search(r"¥{.*¥}", raw, re.S)
            try: loose = json.loads(m.group(0)) if m else None
            except Exception: loose = None
        ok_content = check_content(loose if isinstance(loose, dict) else None, s
        rows.append({"question": ex["question"], "raw": raw, "format_ok": ok_fm
                     "content_ok": ok_content, "both_ok": ok_fmt and ok_content,
    n = len(rows)
    summary = {"label": label, "n": n,
               "format_rate": sum(r["format_ok"] for r in rows)/n,
               "content_rate": sum(r["content_ok"] for r in rows)/n,
               "both_rate": sum(r["both_ok"] for r in rows)/n,
               "seconds": round(time.time()-t0, 1)}
    (WORK / f"eval_{label}.json").write_text(json.dumps({"summary": summary, "r
    print(json.dumps(summary, ensure_ascii=False))
    return summary, rows

before_summary, before_rows = evaluate(model, "before")
# 質問ごとの出力。「正解の needs_review」と見比べ、どのキーが欠けているか、文面
display(pd.DataFrame([{"質問": r["question"], "正解の needs_review": r["gold_ne
                       "形式": mark(r["format_ok"]), "形式の失敗理由": r["forma
```

4. LoRA 学習（T4 で 4〜5 分）

LoRA と QLoRA の違い

- **LoRA**: 基盤モデルの重みは固定し、一部の層に追加した小さな行列（adapter）だけを学習します。
- **QLoRA**: 基盤モデルの重みを 4bit に量子化して保持し、その上で同じように adapter だけを学習します。adapter 自体は量子化しません。

QLoRA の効果は、基盤モデルの重みを保持するメモリが減ることです。16bit の約 4 分の 1 になるので、そのままでは GPU に載らない大きさのモデルでも学習できます（70 億パラメータなら 16bit で約 14GB、4bit で約 3.5GB）。

観点	LoRA（量子化なし）	QLoRA（4bit）
重みのメモリ	大きい	小さい（16bit の約 1/4）
基盤モデルの出力	元のまま	量子化の誤差で変わることがある。影響の大きさはモデルによって違う
学習の速さ	基準	速くはならない。4bit の重みを計算のたびに戻すので、遅くなることが

観点	LoRA（量子化なし）	QLoRA（4bit）
依存ライブラリ	peft	peft に加えて bitsandbytes（CUDA の GPU が前提）

QLoRA を実装するときの注意:

- 読み込み時に `BitsAndBytesConfig(load_in_4bit=True, ...)` を渡し、LoRA を付ける前に `prepare_model_for_kbit_training(model)` を通します。
- 保持精度（4bit）と計算精度（`bnb_4bit_compute_dtype`。16bit など）は別の設定です。
- 学習前後の比較は、同じ量子化条件の基盤モデルで行います。学習前を量子化なし、学習後を 4bit で測ると、量子化の影響と学習の影響が混ざります。
- adapter は学習時の基盤モデルに合わせて作られます。配布や再利用のときは、基盤モデルの ID・revision と量子化設定を一緒に記録します。
- 4bit での学習が勧められないモデルもあります。Qwen3.5 は量子化の誤差が通常より大きいとして、4bit 学習を勧めていない学習ライブラリのガイドがあります。

この演習のモデルは量子化なしでも T4 に載るので、LoRA で学習します。モデルが GPU に載るなら LoRA、載らないなら QLoRA かより小さいモデル、という順で考えます。4bit で読み込んだ場合のメモリと出力の違いは、セクション 8（発展）で確かめられます。

学習の実行

やること: train の 14 件を chat 形式（system / user / assistant）に直し、LoRA を付けて 20 エポック（40 ステップ）学習します。学習した adapter は `adapter/` に保存します。

- `target_modules` は LoRA を差し込む層です。注意機構の射影（`q_proj` など）と MLP の射影（`gate_proj` など）を指定しています。このモデルは 24 層のうち 18 層が線形注意で、`q_proj` などを持つのは残りの 6 層だけです。MLP は全層にあります。
- バッチ 4 × 勾配累積 2 = 1 ステップ 8 件。train は 14 件なので 1 エポックは 2 ステップ（8 件と 6 件）で、20 エポックで 40 ステップです。各例をちょうど 20 回学習します。学習率の warmup と減衰（cosine）はこの 40 ステップに対して効きます。
- 学習後のモデルは変数 `tuned` に入ります。LoRA は基盤モデルの層に差し込まれるので、元の変数 `model` で生成しても adapter 込みの出力になります。学習前の出力が必要なときは、セクション 6 のように adapter を一時的に無効にします。

見るところ

- `trainable params: ... || all params: ... || trainable%: ...`: 学習するパラメータが全体の数 % だけであることが LoRA の特徴です。

- `Training Loss` の表（10 ステップごと）: 1.5 前後から始まり、40 ステップで 0.01〜0.02 まで下がります。下がらないまま終わった場合は、設定を変えていないか確かめてください。
- `学習時間: ... 秒 | ステップ数: 40 | エポック: 20.0`: T4 で 240〜290 秒です。ステップ数とエポックは、学習データ 14 件 × 20 周が上の計算どおりに実行されたことの確認です。

出ても問題ない警告: `The tokenizer has new PAD/BOS/EOS tokens that differ from the model config ...` は、終端トークンの設定を tokenizer に合わせたという案内です。

うまくいかないとき: 学習が途中で止まった、または学習できなかった場合は、下の 4b を読んでください。

```
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig
from datasets import Dataset

def to_chat(example):
    # 学習データは「入力（system + user）」と「望ましい出力（assistant）」の対。
    return {"messages": build_messages(example) + [{"role": "assistant", "conte

train_ds = Dataset.from_list([to_chat(e) for e in train])
valid_ds = Dataset.from_list([to_chat(e) for e in valid])

model.config.use_cache = False
# Qwen3.5 は 24 層のうち 18 層が線形注意で、q/k/v/o_proj を持つのは残りの 6 層。
lora = LoraConfig(r=CONFIG["lora_r"], lora_alpha=CONFIG["lora_alpha"], lora_dro
                  target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_
USE_AMP = False  # 通常は変えない。True にすると計算だけ 16bit の混合精度になる
sft = SFTConfig(output_dir=str(WORK / "ckpt"), num_train_epochs=CONFIG["num_tra
                learning_rate=CONFIG["lr"], lr_scheduler_type="cosine", warmup_
                fp16=USE_AMP, max_length=CONFIG["max_seq_len"], packing=False,
trainer = SFTTrainer(model=model, args=sft, train_dataset=train_ds, eval_datase
trainer.model.print_trainable_parameters()   # LoRA で学習するパラメータの割合
t0 = time.time()
trainer.train()  # 途中で止まった場合の再開は 4b を参照
print(f"学習時間: {time.time()-t0:.0f} 秒 | ステップ数: {trainer.state.global_s
ADAPTER_DIR = WORK / "adapter"
trainer.model.save_pretrained(ADAPTER_DIR)
if USE_DRIVE:
    import shutil; shutil.copytree(ADAPTER_DIR, Path(CONFIG["drive_dir"]) / "ada
tuned = trainer.model
tuned.config.use_cache = True
_ = tuned.eval()  # 学習モードのままだと dropout が効き、評価の出力が揺れる（代.
```

4b. 学習が止まった・できなかった場合

途中で止まった（ランタイムの切断など）：チェックポイントは 10 ステップごとに `/content/glora_lab/ckpt/` に保存されています。ファイルが残っていれば、セクション 0〜3 を実行し直してから、上の学習セルの `trainer.train()` を `trainer.train(resume_from_checkpoint=True)` に書き換えて実行すると、最後のチェックポイントから 40 ステップ（20 エポック）まで続きを学習します。ランタイムごと消えてファイルが無い場合は、学習をやり直すか、下の方法に切り替えます。

学習できなかった（GPU が確保できない、時間内に終わらない）：講師が同じノートブックを T4 で実行した結果（学習後の出力とスコア）を使って、比較と考察を進めます。セクション 5 以降のセルは学習後のモデル `tuned` が無いと動かないので、講師の結果を見ながら説明を読んでください。提出するときは、セクション 7 の `executed_training_myself` を `False` にし、本人が実行できた範囲を明記します。

5. 学習後評価（学習前と同じ test・同じ生成設定）

やること: 学習後のモデル `tuned` で、学習前とまったく同じ 21 問・同じ生成設定で評価し、前後を並べます。変えたのは adapter の有無だけです。

見るところ

- 比較表: `format_rate`（形式遵守率）と `content_rate`（内容正解率）が学習前からどう変わったか。
- `退行`: 学習前は形式も内容も通っていたのに、学習後に落ちた質問です。空のリストが期待値です。
- 質問ごとの表: 残った失敗を読み、どの型かを見分けてください。
  - 根拠なしの質問（正解の `needs_review` が `True`）に、FAQ の別項目を当てはめて断定した。
  - 根拠ありの質問（正解が `False`）に、「記載がありません」と確認要求で逃げた。

学習後の回答は、教師データの文（FAQ の原文）をほぼそのまま返す形になります。形式と文面は揃いますが、「FAQ に根拠があるか」の判断まで学習できたとは限りません。

seed を固定しても、GPU 上の学習は完全には再現しません。形式遵守率と内容正解率の割合はほぼ同じでも、どの質問が残るかは実行ごとに変わることがあります。隣の人と「残った質問」が違っても異常ではないので、失敗の型で比べてください。

```
after_summary, after_rows = evaluate(tuned, "after")
compare = pd.DataFrame([before_summary, after_summary]).set_index("label")[["n",
display(compare)
# 学習前は形式も内容も通ったのに、学習後に落ちた質問（退行）
regressed = [b["question"] for b, a in zip(before_rows, after_rows) if b["both_
print("退行:", regressed)
```

```
7
8 # 質問ごとの出力原文。形式（JSON・必須キー・型）と内容（needs_review の一致と必
9 detail = pd.DataFrame([{"質問": b["question"], "正解の needs_review": b["gold_n
10                       "学習前の出力": b["raw"], "学習前 形式/内容": f"{mark(b
11                       "学習後の出力": a["raw"], "学習後 形式/内容": f"{mark(a
12                      for b, a in zip(before_rows, after_rows)])
13 display(detail)
```

6. 知識を問う質問（形式を学んでも知識は保証されない）

やること: 形式の実験とは別に、FAQ にも学習データにも無い事実を 3 問尋ねます（対照として FAQ にある質問を 1 問加えます）。望ましい応答は「確認が必要（`needs_review: true`）」です。

応答を 3 つに分類します。

分類	意味
`format_error`	JSON の形式を守れていない（学習前に多い）
`needs_review`	「記載がありません。確認が必要です」と返した（望ましい応答）
`asserted`	`needs_review: false` で言い切った。FAQ に無い質問に対してなら、根拠のない断定（創作）で

見るところ: 学習前は形式違反になりやすく、学習後は「確認が必要」と「断定」の両方が出ることがあります。出力の原文も表に出すので、断定した場合に何を根拠にしたように見えるか（FAQ の別項目を流用しているか）を読んでください。「Fine-Tuning では知識は絶対に直らない」と先に結論にせず、今回のデータでの結果を報告します。

学習前の出力は、adapter を一時的に無効にして取っています（`tuned.disable_adapter()`）。

```
1 KNOWLEDGE_QUESTIONS = [
2     "この講座の最終課題の締切はいつですか",        # FAQ に無い（架空）
3     "担当教員の名前は何ですか",                    # FAQ に無い
4     "次回の講義のテーマは何ですか",                # FAQ に無い
5     "課題の提出期限はいつですか",                  # FAQ にある（対照）
6 ]
7 def classify(raw):
8     ok, _, obj = check_format(raw)
9     if not ok or obj is None: return "format_error"
10     return "needs_review" if obj["needs_review"] else "asserted"
11
12 knowledge_rows = []
13 for q in KNOWLEDGE_QUESTIONS:
14     ex = make_example(q, "", True)
15     # LoRA は基盤モデルの層に差し込まれるので、学習後は `model` で生成しても ada
16     with tuned.disable_adapter():
17         before_raw = generate(tuned, ex)
18     after_raw = generate(tuned, ex)
19     knowledge_rows.append({"質問": q, "FAQ にある": q in gold, "学習前の分類": c
```

```
20                  "学習後の分類": classify(after_raw), "学習後の出力": after_raw
21 display(pd.DataFrame(knowledge_rows))
```

7. 比較表と提出

やること: 設定、ライブラリの版、学習前後の要約、退行した質問を 1 つの `result.json` にまとめます。

提出するもの

- `result.json`、`eval_before.json`、`eval_after.json`（`/content/qlora_lab` にあります。左のファイル一覧で右クリック → ダウンロード。`USE_DRIVE = True` なら Drive にもコピーされます）
- セクション 5 の比較表と、残った失敗の説明（どの型の失敗か）
- セクション 6 の結果（確認要求に変わった応答、断定が残った応答）
- 本人が学習を実行した範囲。講師の結果を使った場合は `executed_training_myself` を `False` にしてください。

```
 1 result = {
 2     "config": {k: v for k, v in CONFIG.items() if k not in ("work_dir", "drive_
 3     "versions": {m.__name__: m.__version__ for m in [transformers, peft, trl, t
 4     "method": "LoRA（量子化なし）",
 5     "before": before_summary, "after": after_summary,
 6     "regressed": regressed,
 7     "knowledge": knowledge_rows,
 8     "executed_training_myself": True,   # 講師の結果を使った場合は False にする
 9 }
10 (WORK / "result.json").write_text(json.dumps(result, ensure_ascii=False, indent=
11 print(json.dumps({k: v for k, v in result.items() if k != "knowledge"}, ensure_a
12 if USE_DRIVE:
13     import shutil
14     for name in ["result.json", "eval_before.json", "eval_after.json", "test.js
15         shutil.copy(WORK / name, Path(CONFIG["drive_dir"]) / name)
16     print("Drive に保存しました:", CONFIG["drive_dir"])
```

8. 発展: 4bit 量子化で読み込み、メモリと出力を比べる（任意）

やること: 同じモデルを 4bit（NF4）で読み込み、重みのメモリと、学習前の出力がどう変わるかを見ます。QLoRA は、この 4bit の基盤モデルに LoRA を付けて学習する方法です。ここでは学習はしません。

- bitsandbytes は CUDA の GPU が前提なので、GPU ランタイムでだけ動きます。導入に 1 分ほどかかります。

- 4bit の読み込みは `device_map="auto"` で GPU に配置します（bitsandbytes の読み込み方式に合わせています）。
- 4bit 側で学習まで試す場合は、`prepare_model_for_kbit_training(model_4bit)` を通してからセクション 4 と同じ手順で学習し、学習前後とも 4bit の基盤モデルで評価します。

見るところ

- 重みのメモリ: 32bit の約 3GB に対して 4bit は 1GB 未満になります。
- 比較表と質問ごとの表: 学習していないので、32bit の学習前（`before`）との差は量子化の影響です。良くなる方向に変わる質問もあれば、`needs_review` が `"false"`（文字列）になって型検査で落ちる、根拠なしの質問に「はい」「なし」と断定する、といった出力も見られます。方向が一定でないことを確かめてくだ

```
%pip install -q "bitsandbytes==0.50.*"
from transformers import BitsAndBytesConfig

bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",    # 保持
                         bnb_4bit_compute_dtype=torch.float16,              # 計
                         bnb_4bit_use_double_quant=True)
model_4bit = AutoModelForCausalLM.from_pretrained(CONFIG["base_model"], revisio
print("重みのメモリ(GB) | 量子化なし（32bit）:", BASE_MEMORY_GB, "| 4bit:", rou

# 学習はしていないので、量子化なしの学習前（before）との差が量子化の影響
q_summary, q_rows = evaluate(model_4bit, "before_4bit")
display(pd.DataFrame([before_summary, q_summary]).set_index("label")[["n", "for
display(pd.DataFrame([{"質問": r["question"], "正解の needs_review": r["gold_ne
                       "形式/内容": f"{mark(r['format_ok'])}/{mark(r['content_ol
```